Homotopical Encrypt: Hidden Hodge Operators and the Harmonic Cochain Problem

Chloe Vander Vlugt

Abstract

We introduce the Harmonic Cochain Problem (HCP), a hidden-operator recovery problem in which public cochain or class-level data does not determine the operative Hodge–Laplacian kernel. In the Homotopical Encrypt model, public samples have the form $b = h + e$, where $h \in \ker(\Delta)$ is a harmonic cochain and $e \in \mathrm{im}(\delta) \oplus \mathrm{im}(\delta^*)$ is Hodge-structured noise. We compare HCP with the closest vector problem (CVP): CVP is the known-lattice comparison problem, while HCP hides the operator and kernel that make the relevant lattice explicit. We also describe a certified projection chain from accepted hidden/geometric state to a finite Reed–Solomon mutual-correlated-agreement target, with theorem boundaries formalized in Lean.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Contributions . . . 3

2 The Harmonic Cochain Problem **4**
- 2.1 Public And Hidden Data . . . 4
- 2.2 Recovery Target . . . 5
- 2.3 Computational Model . . . 5

3 CVP Comparison And Kernel Ambiguity **6**
- 3.1 Reduction Direction . . . 6
- 3.2 Oracle Insufficiency . . . 6
- 3.3 Structural Comparison . . . 6

4 Axioms, Semantics, And Public Data **7**
4.1 Fibrational Semantics . . . 7
4.2 Fibrational Identity Objects . . . 7
4.3 Homotopy Classes . . . 7
4.4 Axiom Surfaces . . . 7
4.5 Exact Structural Invariants . . . 8
4.6 Heyting And Oracle Judgment Surface . . . 8

5 Public Artifact Stack And Lossy Projection **9**
5.1 Public And Private Data Handling . . . 9
5.2 The Artifact Stack . . . 9
5.3 Cochain Commitments . . . 10
5.4 Commitment Map . . . 10
5.5 Commitment And Artifact Invariants . . . 10
5.6 Sheaf Storage And Witness Material . . . 11
5.7 Lossy Projection . . . 11
5.8 Preserved And Lost Structure . . . 11
5.9 Fundamental Properties . . . 12

6 Reed–Solomon Projection And MCA **12**
6.1 Projection Chain . . . 13
6.2 Projection-Stage Gates . . . 13
6.3 Canonical Reed–Solomon Endpoint . . . 14
6.4 Mutual Correlated Agreement . . . 14
6.5 Corollary . . . 15

7 Formal Verification **15**
7.1 Lean Formalization . . . 15
7.2 Repository Structure . . . 15
7.3 Theorem Correspondence . . . 16

8 Future Directions **17**
8.1 Cryptographic Protocol . . . 17
8.2 Anonymous Identity System . . . 17
8.3 Sheaf Storage . . . 18
8.4 Vertra Chain . . . 18
8.5 Complexity-Theoretic Extensions . . . 18

A Repository Dependency Graph **18**

B Lean/Paper Correspondence **19**
B.1 Definitions And Interfaces 19
B.2 Theorems And Status 20

1 Introduction

1.1 Motivation

Homotopical Encrypt is organized around the following security shape:

$$\text{public cochain/shadow data} \quad \not\Rightarrow \quad \text{the operative Hodge–Laplacian kernel.}$$

The private recovery object is not merely a scalar secret or a vector in a published lattice. It is the harmonic cochain selected by a hidden Hodge operator. Public verification therefore needs finite artifacts, while security must not collapse to recovering the private representative or the hidden operator. The resulting design goal is transparency in action and anonymity in identity: public artifacts should make accepted actions auditable, without making the hidden representative or identity-selecting kernel public.

1.2 Contributions

This paper develops the following contributions.

(i) The Harmonic Cochain Problem (HCP), a hidden-operator recovery problem for Hodge-theoretic cochains.

(ii) A Hodge-structured noise model with public samples $b = h + e$, where $h \in \ker(\Delta)$ and $e \in \operatorname{im}(\delta) \oplus \operatorname{im}(\delta^*)$.

(iii) A comparison with CVP, separating known-lattice closest-vector search from hidden-operator harmonic-cochain recovery.

(iv) A projection framework from hidden/geometric state to public finite evidence.

(v) A Lean formalization of the definitions, reductions, projection chain, object-local invariants, projection-stage certificates, and theorem boundaries.

(vi) A Reed–Solomon mutual-correlated-agreement corollary aligned with the vocabulary of the Proximity Prize problem statement.

2 The Harmonic Cochain Problem

This section states the central search problem and the hidden-operator model in which it is used.

Definition 2.1 (Harmonic Cochain Problem)**.** *The* Harmonic Cochain Problem *(HCP) asks, given public cochain/class-level data associated to a Hodge system, to recover the harmonic cochain determined by the operative Laplacian kernel.*

In the Homotopical Encrypt public-sample model, the visible cochain has the form

$$b = h + e, \qquad h \in \ker(\Delta), \qquad e \in \operatorname{im}(\delta) \oplus \operatorname{im}(\delta^*).$$

Here h is the harmonic cochain, equivalently the global-section component, and e is Hodge-structured noise. The hidden-operator condition is that the public data does not determine a unique Δ or $\ker(\Delta)$.

2.1 Public And Hidden Data

Definition 2.2 (Public observation)**.** *A public observation is the data exposed to a verifier or adversary before choosing a hidden representative. In the abstract model it consists of a public class or shadow value p, together with cochain-facing data b. In the Homotopical Encrypt instantiation, this public layer is represented by public shadows, cochain commitments, topology hashes, oracle judgments, and proof-carrying artifacts.*

The public observation is intentionally not the hidden Hodge system. It is a classifier of a public equivalence class of possible hidden representatives. This is the point at which the fibrational and homotopical semantics enter the model: public equality is equality after applying a public classifier, not strict equality of hidden representatives. Section 4 develops this semantic layer.

Definition 2.3 (Hidden Hodge witness)**.** *A hidden Hodge witness for a public observation (p, b) consists of hidden operator data Δ, a harmonic cochain h, and Hodge-structured noise e, such that*

$$\Delta \in \mathsf{Hidden}(p), \qquad h \in \ker(\Delta), \qquad e \in \operatorname{im}(\delta) \oplus \operatorname{im}(\delta^*), \qquad b = h + e.$$

Here $\mathsf{Hidden}(p)$ denotes the fiber of hidden Hodge–Laplacian representatives compatible with the public class value p.

Concretely, the hidden data includes the fibration representative, restriction maps, the induced Laplacian, its harmonic kernel, and private witness/cochain data. The public data may bind to this hidden data through commitments or proof-carrying artifacts, but it does not present the operative kernel itself.

Definition 2.4 (HCP solution)**.** *A solution to an HCP instance* (p, b) *is a cochain* h' *that is the harmonic component selected by the operative hidden witness. In the exact recovery form used by the hidden-operator separation,* $h' = h$ *for every hidden witness* (Δ, h, e) *compatible with the public observation.*

The exact recovery formulation is deliberately strong. It isolates the ambiguity proved in the Lean development: the same public cochain may be compatible with hidden operators that disagree about harmonicity. Thus a solver that sees only public data cannot be uniformly sound unless the public observation determines enough hidden-operator structure.

2.2 Recovery Target

The recovery target is the harmonic cochain selected by the operative kernel, not necessarily the entire kernel subspace. This distinction keeps HCP parallel to CVP: CVP asks for a closest vector in a presented lattice, while HCP asks for a harmonic cochain in the operative Hodge kernel.

In the metric variant used for comparison with CVP, one may ask for the closest harmonic cochain to b. Once Δ or a basis for $\ker(\Delta)$ is explicitly presented, this becomes a closest-vector question on the harmonic lattice. The hidden-operator model withholds precisely that presentation.

2.3 Computational Model

The computational model separates three layers.

(i) The mathematical layer defines public projections, hidden operator fibers, harmonicity, and kernel ambiguity.

(ii) External hardness assumptions and cited coding-theory theorems enter at explicitly labeled theorem boundaries.

(iii) Runtime obligations such as byte parsing, canonical encodings, hash domain separation, proof verification, and state transition are outside the mathematical HCP definition.

This paper focuses on the first two layers. The runtime layer is part of the protocol and proof-carrying-verification story developed in the sequel.

3 CVP Comparison And Kernel Ambiguity

CVP asks for a closest vector in an explicitly presented lattice. HCP asks for a harmonic cochain in the operative Hodge kernel, and in the Homotopical Encrypt model that kernel is not explicitly presented by public data.

3.1 Reduction Direction

Proposition 3.1 (HCP solver exposes known-lattice CVP)**.** *In the formal model, a solver that recovers harmonic components for every presented Hodge operator induces a solver for the corresponding explicit closest-vector problem on the harmonic lattice.*

This is the direction formalized in Lean by the Hodge/CVP reduction surface: once the hidden operator data has been fixed or presented, the closest harmonic cochain problem becomes a CVP-style problem on $\ker(\Delta)$. Thus CVP is a known-lattice shadow of the Hodge recovery problem.

3.2 Oracle Insufficiency

Proposition 3.2 (Known-lattice CVP oracle insufficiency)**.** *In the hidden-operator model, an oracle for explicit known-lattice CVP does not by itself determine the operative Hodge–Laplacian kernel. Consequently, such an oracle does not uniformly solve HCP from public data alone.*

The obstruction is kernel ambiguity: the same public observation may be compatible with multiple hidden operators whose harmonic kernels differ. A CVP oracle can solve a closest-vector problem after the lattice is presented; it does not select which hidden kernel public data was meant to present.

3.3 Structural Comparison

The comparison separates three surfaces:

(i) known-lattice closest-vector search;

(ii) hidden-operator HCP;

(iii) public projection/proximity verification.

The first two surfaces are the hardness story. The third is a verifier-facing theorem: given accepted hidden/geometric data, later sections construct finite public shadows and an RS/MCA endpoint without claiming that the projection is injective.

4 Axioms, Semantics, And Public Data

This section fixes the semantic and axiomatic surfaces used by the problem. It comes before the public artifact and commitment layer because those objects are only meaningful after the public/private boundary is fixed.

4.1 Fibrational Semantics

The semantic layer is motivated by Helfer's first-order homotopical logic [Hel19]. In this paper, the corresponding Lean files are used as a formalized interface for the semantic structure needed by Homotopical Encrypt, not as a claim that every external theorem from Helfer's paper has been reproved.

4.2 Fibrational Identity Objects

Public equality is not strict equality of hidden representatives. It is mediated by public classifiers, path data, and equivalence of the observable structure. This is the semantic source of the hidden-operator model used in HCP.

4.3 Homotopy Classes

Public shadows may identify multiple hidden representatives while preserving the invariants required by a verifier. At this stage the shadow is only a public classifier or fingerprint; finite-code projection is introduced later.

4.4 Axiom Surfaces

The Lean development separates assumptions and theorem surfaces into small modules. The relevant early interfaces are:

(i) foundational cochain and Hodge assumptions;

(ii) fibration and homotopy-class interfaces;

(iii) Heyting/oracle semantics for public judgments;

(iv) sheaf-coherence and spectral interfaces;

(v) cochain-commitment payload and security assumptions.

4.5 Exact Structural Invariants

The exact gates used by the Lean development are not a separate source of structure. They package the same foundational and sheaf-coherence hypotheses at the point where an object is admitted. In Lean, `ExactGateBundle` is the foundation interface and `SheafExactGateBundle` is the sheaf-coherence interface. The theorem `structural_preconditions_of_sheaf_exact_gates` extracts the two structural consequences used later: the cochain differential squares to zero and the Hodge Laplacian is the one induced by the coboundary data.

Proposition 4.1 (Exact structural preconditions)**.** *An accepted sheaf-coherence surface supplies*

$$d^2 = 0 \qquad \textit{and} \qquad \Delta = d^*d + dd^*$$

for the cochain complex under discussion.

4.6 Heyting And Oracle Judgment Surface

The public judgment layer is also fixed here, because HCP uses public classifiers and oracle-visible predicates before any finite-code projection is introduced. Lean separates an `OracleQuery` from transaction cochain commitments: its `id_commit` is an identity hash obtained from the public Hodge projection and quantization path, not a cochain commitment payload.

The oracle predicate checks public compatibility data such as the vertex identifier, Betti number, identity hash, and Tsirelson-style spectral judgment. The induced judgment is then mapped into the three-valued Heyting surface used by the acceptance predicate. The stability axiom `OracleEta2` records that equal public identity inputs produce the same identity hash; it is a determinism condition for public identity observations, not a uniqueness or binding theorem for cochain commitments.

This modular split is part of the mathematical statement. It prevents runtime facts, cryptographic assumptions, public classifiers, and Hodge-theoretic claims from being conflated. Exact and oracle invariants are used here as mathematical acceptance surfaces. Consensus composition and chain/runtime enforcement are recorded only at the formalization boundary; their full CS/protocol treatment is deferred to the sequel.

5 Public Artifact Stack And Lossy Projection

Commitments and public shadows are the first concrete public-facing objects in the paper. They explain what public data can carry without revealing the hidden Hodge representative. They are the scaffolding for transparency in action and anonymity in identity: actions are exposed through auditable public artifacts, while identities remain inside hidden representatives and public equivalence classes.

5.1 Public And Private Data Handling

The public layer may contain commitments, topology hashes, roots, public shadows, oracle judgments, and proof-carrying artifacts. The private layer contains the fibration representative, transport/restriction data, full cochains, and the Hodge operator whose kernel selects the harmonic component. The paper's claims are stated so that public artifacts bind to hidden witnesses without turning the hidden witness into public data.

5.2 The Artifact Stack

The public artifact stack has the following shape:

hidden/private object
$\rightarrow$ public shadow or compressed public datum
$\rightarrow$ commitment/root/hash artifact
$\rightarrow$ proof-carrying artifact
$\rightarrow$ verifier judgment.

Each layer binds to or summarizes the preceding layer. A hash does not replace the mathematical object, a commitment does not replace the witness, and a proof artifact does not replace the proof obligation. The purpose of the stack is to let public verification refer to hidden sheaf/cochain data without exposing it. In this sense, the stack separates what must be publicly accountable from what must remain representative-private.

Hashes serve as public names, integrity bindings, and fixed-size pointers for larger artifacts. A topology hash names the public combinatorial support; a root commits to a batch or storage view; a transcript hash domain-separates a proof statement. None of these hashes asserts validity by itself. Validity comes from the predicate or proof attached to the hashed object.

5.3 Cochain Commitments

Cochain commitments provide the public handle on hidden cochain data. The Lean surface treats the canonical payload as a mathematical object with a modulus, edge values, declared edge count, and topology hash.

5.4 Commitment Map

Definition 5.1 (Cochain commitment payload)**.** *A cochain commitment payload is a tuple*

$$(q, n, c, \tau)$$

consisting of a modulus q, a declared edge count n, a list c of integer edge values, and a topology hash τ. In the canonical Lean payload this is `CochainCommitmentData`*.*

The intended commitment map is the transaction-facing coboundary map

$$(m, r) \longmapsto \delta(m \| r) \in C^1 \pmod q,$$

stored together with the topology hash of the supporting complex. The payload therefore preserves the accepted modulus, the edge-vector shape, the public topology binding, and the serialized C^1-level commitment values. It does not expose the hidden fibration representative, private transport data, the operative Hodge–Laplacian, or the harmonic kernel.

Definition 5.2 (Commitment well-formedness)**.** *A cochain commitment payload is well formed when its modulus is the supported canonical modulus, its declared edge count agrees with the actual edge-value list length, and the edge list is nonempty.*

The security interface for this commitment layer is `CochainCommitmentAxioms`. It contains binding, hiding, homomorphic addition, homomorphic scalar multiplication, topology binding, and partial-opening soundness. The blinded-commitment, vector-commitment, and cohomological-MAC interfaces are separate axiom records rather than consequences silently bundled into the payload shape.

5.5 Commitment And Artifact Invariants

The commitment-facing invariants attach to the commitment payload and to the public artifact stack, not to the proximity projection. Payload well-formedness is a shape condition: it asserts the supported modulus, agreement

between the declared edge count and the edge-value list, and a nonempty edge list. It is not a cryptographic uniqueness theorem and it does not assert that the cochain vector is nonzero. For example, the canonical zero payload is well formed whenever the declared edge count is positive.

Topology compatibility is a separate public invariant. Two committed cochains can be aggregated only when their moduli, declared edge counts, and topology hashes agree. This is the Lean predicate `AddCompatible`; the weaker same-topology predicate records equality of topology hashes alone. The cryptographic binding and topology-binding claims are supplied by `CochainCommitmentAxioms`, not by the raw payload record.

At the artifact level, Lean records a hashed committed cochain as a public cochain payload together with digest data and a binding predicate. A proof-carrying committed cochain adds a proof system, a statement, and a verification predicate. The basic theorem exposed to this paper is that a well-formed proof-carrying committed-cochain artifact has an underlying well-formed cochain commitment payload.

5.6 Sheaf Storage And Witness Material

Storage roots, topology hashes, cochain roots, nullifiers, and proof objects are public artifacts in the stack. Full sheaf, cochain, and fibration data remain witness material unless a protocol path explicitly exposes them. The storage mechanics are protocol-level concerns; the mathematical point here is that stored public artifacts are shadows of accepted witness data, not replacements for the hidden object.

5.7 Lossy Projection

Compression creates verifier-facing public artifacts from hidden or full witness data. The compression need not be injective; it must preserve enough structure to support the relevant public statement.

A public shadow is allowed to forget hidden data. In fact, it must forget enough data that it does not select a private representative. The relevant question is not whether the public datum is injective, but which facts survive the compression relation.

5.8 Preserved And Lost Structure

The preserved structure before the finite-code layer is:

(i) public classifier or fingerprint equality for hidden representatives;

(ii) cochain commitment payload well-formedness;

(iii) topology binding for cochain commitments;

(iv) digest/root binding for compressed public shadows;

(v) proof-carrying validity of the public statement attached to an artifact.

The lost structure is the private representative itself: full sheaf/cochain witness data, hidden transport, the operative Hodge operator, private randomness, and runtime lifecycle facts. Those may be bound to public artifacts or certified by a proof-carrying statement, but they are not made public merely by the presence of a hash, root, or commitment payload.

5.9 Fundamental Properties

Proposition 5.3 (Well-formed committed payloads)**.** *A well-formed cochain commitment payload has nonempty edge data and modulus greater than one.*

Proposition 5.4 (Topology-compatible aggregation)**.** *Commitment aggregation is only meaningful for payloads with compatible modulus, declared edge count, and topology hash. This is the public topology-binding condition consumed by the chain layer.*

Proposition 5.5 (Proof-carrying artifact exposes payload)**.** *A well-formed proof-carrying committed-cochain artifact exposes a well-formed underlying cochain commitment payload.*

Byte parsing, allocation bounds, canonical encoding, concrete hash functions, domain separation, storage inclusion proofs, proof verification, state updates, and ephemeral-fiber lifecycle properties are TCB obligations. Lean records their boundary predicates; it does not identify them with the mathematical cochain commitment payload.

6 Reed–Solomon Projection And MCA

The concrete projection chain belongs here because its endpoint is the Reed–Solomon received-word surface and the mutual-correlated-agreement theorem.

6.1 Projection Chain

The staged chain has the following shape:

FrameWitness
→ Projective Plucker shadow
→ committed cochain/local syndrome
→ plane shadow
→ finite-field received word
→ canonical Reed–Solomon target.

Each stage has local proximity gates and certificates. The orchestrating projection-chain theorem composes these local certificates without rechecking them. These are proximity-specific projection gates, not the main Homotopical Encrypt acceptance layer from Section 4. Their role is to certify the lossy path into the Reed–Solomon endpoint after the main hidden/geometric object has already been accepted.

6.2 Projection-Stage Gates

The source and Plucker stages carry metric and public-shadow invariants. `SourceMetricGate` records nonnegativity and quotient-invariance of the source disagreement measure. `PluckerMetricGate` records the admissible chart, projective-distance, valid-frame, nonnegativity, and quotient-invariance conditions. `PublicShadowRespectGate` says that equality of the public shadow determines the same committed cochain, plane shadow, and received word along this projection path.

The Plucker-to-cochain stage carries the local-syndrome interface. A concrete `PayloadEncoder` sends valid Plucker frame data to a well-formed cochain commitment payload with the expected edge count. The `SyndromeAdequacyGate` then states that a well-formed committed cochain with zero local syndrome is a global-section candidate for the chosen sheaf. In the concrete bridge, the local syndrome is the commitment-sum witness; the Lean theorem records that global candidates have zero commitment sum.

The cochain-to-plane and plane-to-RS stages carry semantic-alignment and non-collapse conditions. `SemanticAlignmentGate` requires the valid frames, plane shadow, and received word to agree with the extraction maps. `PlaneExtractionContext.Aligned` records the cochain well-formedness, expected edge count, and selected-index bounds needed for plane extraction. The far-region gates `PluckerToCochainNonCollapseGate`,

`CochainToRSFarGate`, and `FarGatePipeline` provide the calibrated non-collapse evidence consumed by the final RS/MCA corollary.

6.3 Canonical Reed–Solomon Endpoint

The proximity endpoint is stated over the paper-style alphabet shape $C \subseteq (\mathbb{F}^s)^n$. The canonical Homotopical Encrypt evaluation is the ordinary Reed–Solomon slice at width $s = 1$.

6.4 Mutual Correlated Agreement

Definition 6.1 (MCA bad-challenge fraction)**.** *Let $C \subseteq (\mathbb{F}^s)^n$ be a block code and let $\ell(\gamma) = f_1 + \gamma f_2$ be an affine line of received words. For a radius δ, the bad challenges are those $\gamma \in \mathbb{F}$ for which $\ell(\gamma)$ is δ-close to C on a large coordinate set, but the pair (f_1, f_2) is not correspondingly close to the two-fold interleaved code $C^{\oplus 2}$ on that same set. The MCA error is the fraction of bad challenges in $\mathbb{F}$.*

The Lean endpoint specializes this paper definition to the canonical scalar Reed–Solomon slice $s = 1$. The width-one comparison is formalized in `CanonicalRS.MCA.Concrete`: ordinary scalar RS words are identified with words in $(\mathbb{F}^1)^n$, and the bad-challenge predicate is shown to agree with the paper-alphabet predicate.

Theorem 6.2 (Canonical RS MCA bound)**.** *For the canonical Homotopical Encrypt Reed–Solomon endpoint at the below-Johnson radius, every projection-induced final affine line has bad-challenge fraction at most*

$$\frac{\tau}{|\mathbb{F}|},$$

where τ is the cited MCA threshold for the canonical endpoint.

The deep list-decoding and MCA estimates are not reproved in Lean. They enter through `CanonicalRS.CitedBounds` as named theorem constants: the Johnson list-decoding bound, the count and fractional MCA bounds, and the paper Definition 4.3 width-one MCA bound. Lean proves the endpoint plumbing, the $s = 1$ equivalence, and the specialization to projection-induced lines.

6.5 Corollary

Corollary 6.3 (Verifier-facing RS/MCA projection)**.** *Given an accepted hidden/geometric witness, a certified Plucker-to-cochain-to-plane-to-RS projection chain, the required hard projection-gate evidence, and a canonical Johnson-radius far-separation witness, the public verifier obtains a finite Reed–Solomon received word whose canonical MCA bad-challenge fraction is bounded by the cited threshold* $\tau/|\mathbb{F}|$*.*

This corollary is a proximity statement. It says that accepted hidden/geometric state can be projected to a finite public RS/MCA target with calibrated non-collapse at the selected far region. It does not say that the projection is injective, and it does not turn the RS/MCA endpoint into the source of HCP hardness.

7 Formal Verification

7.1 Lean Formalization

The Lean development records the mathematical predicates, reductions, projection stages, local certificates, and trusted theorem boundaries used by the paper. The proof is not a proof of the Rust runtime; runtime parsing, canonical encodings, domain separation, and proof verification are handled by a separate TCB boundary.

7.2 Repository Structure

The formalization is split by theorem surface.

(i) `Axioms` contains the foundation, fibration, sheaf, Heyting, oracle, commitment, spectral, and stage-admissibility interfaces.

(ii) `Laplacian`, `CVP`, and `Hardness` contain the projection-model Hodge facts, explicit-lattice CVP comparison, public projection ambiguity, and Hodge/CVP theorem surfaces.

(iii) `Gates` contains gate-facing aliases and lemmas for the exact, spectral, oracle, and stage invariants stated with the corresponding mathematical objects.

(iv) `Proximity` contains the Plucker shadow, cochain bridge, plane projection, Reed–Solomon endpoint, projection chain, and proximity-specific gate certificates.

(v) `Proximity.CanonicalRS` contains the canonical RS endpoint, bridge data, cited bounds, and MCA specialization.

(vi) `TCB` contains proof-carrying artifacts, compression boundaries, observation boundaries, SNARK specialization, witness verification, and runtime-owned obligations.

(vii) `Chain` contains the chain-level transaction, block, ledger, and acceptance predicates that consume these surfaces in the sequel's protocol setting.

7.3 Theorem Correspondence

The core correspondence is:

HCP hidden-kernel ambiguity.
`Hardness.KernelAmbiguous`; Lean-proved surface.

Lossy public projection.
`Hardness.LossyPublicProjection`; Lean-proved implication.

Hodge projection as CVP.
`Hardness.Hodge_Projection_Is_CVP`; Lean-proved.

Exact structural preconditions.
`Gates.Exact` and `Gates.structural_preconditions_of_sheaf_exact_gates`; alias/proved interface.

Oracle/Heyting judgment surface.
`Axioms.OracleModality`; assumption/interface.

Cochain payload well-formedness.
`CochainCommitmentData.WellFormed`; Lean-proved predicates.

Proof-carrying artifact.
`ProofCarryingCommittedCochain.WellFormed`; TCB boundary.

Proximity gate certificates.
`Proximity.Gates`; stage-local interfaces.

Projection chain.
`ProjectionChain.Chain.toEvaluationProjection`; Lean-proved plumbing.

Canonical RS bridge data.
`CanonicalRS.BridgeData`; Lean-proved endpoint facts.

RS list/MCA bounds.
`CanonicalRS.CitedBounds`; cited external theorem boundary.

Canonical MCA corollary.
`CanonicalRS.MCA.projectionBadChallengeFraction_le`; Lean-proved from cited bounds.

The status labels are part of the verification claim. Lean-proved rows are formal consequences of the imported modules. Cited external theorem rows are trusted mathematical inputs from the coding-theory literature. TCB rows are runtime obligations about parsing, encoding, hashing, proof verification, and state transition. Assumption/interface rows are explicit predicates whose implementations or external justifications are not silently folded into the core HCP theorem.

8 Future Directions

8.1 Cryptographic Protocol

The present paper stops at the mathematical and formalization boundary. A protocol paper should turn the commitment, oracle, proof-carrying, and proximity surfaces into explicit adversary games and executable verification paths. The relevant Lean surfaces are already separated: `TCB` records proof-carrying artifacts and runtime obligations, `Gates` records object-local admissibility predicates, and `Consensus` records the Proof-of-Value scoring and policy layer. The concrete protocol must specify which certificates are public evidence, which witnesses remain private, and which runtime failures cause rejection, hold, or re-evaluation.

8.2 Anonymous Identity System

The hidden-representative/public-classifier split suggests an anonymous identity architecture. The public identity surface would expose class values, hashes, and oracle judgments, while the private side would retain fibration representatives, transport data, and the Hodge operator selecting the operative kernel. In Lean, this direction is anchored by `Axioms.HomotopyClass`, `Axioms.Fibration`, and `Axioms.OracleModality`. A future system should make the unlinkability, class-membership, and nullifier properties explicit rather than treating public-shadow equality as strict identity.

8.3 Sheaf Storage

The commitment layer can also be developed as a storage and retrieval system for controlled sheaf/cochain projections. The public store would retain topology hashes, cochain roots, compressed shadows, and proof-carrying statements. Full sheaf and fibration witnesses would remain private or be opened only through dedicated proof paths. The relevant Lean interfaces are `Axioms.CochainCommitment`, `TCB.CompressionBoundary`, `TCB.ObservationBoundary`, and `Proximity.Public`.

8.4 Vertra Chain

Vertra is the working name for the chain/network layer. The paper should keep the mathematical protocol name separate from product and token naming. The chain layer belongs in the sequel: it consumes the mathematical surfaces from this paper as protocol evidence, but it also requires block validity, ledger updates, fork choice, oracle attestation, worker reliability, and runtime trust-boundary analysis. Lean files such as `Chain`, `Consensus`, and `Consensus.ProofOfValue` already mark this boundary without making those architecture claims part of the HCP theorem.

8.5 Complexity-Theoretic Extensions

The current formalization isolates the hidden-operator obstruction: public data does not canonically determine the operative Hodge–Laplacian kernel. Future work should refine this into sharper complexity assumptions, adversary classes, and oracle-separation statements. Natural targets include quantitative families of hidden kernels, reductions between HCP variants, leakage models for public classifiers, and a clearer boundary between external CVP assumptions and project-native hidden-operator ambiguity.

A Repository Dependency Graph

The Lean formalization is organized so that each paper section has a small number of import surfaces.

HCP and CVP.
`Axioms/*` feeds `Laplacian/*`, `CVP`, and `Hardness`.

Commitments and public artifacts.
`Axioms.CochainCommitment/*` feeds `TCB/*` and `Proximity.CochainBridge`.

Projection chain.
`Proximity.GrassmannianPlucker` feeds `Proximity.PluckerCochain`, then `Proximity.CochainPlane`, then `Proximity.CochainReedSolomon`.

Projection certificates.
`Proximity.ProjectionChain` feeds `Proximity.Gates`, which feeds `Proximity.CanonicalRS`.

Protocol boundary.
`TCB/*`, `Gates/*`, `Consensus/*`, and `Chain` are the sequel-facing protocol surfaces.

For Paper 1, the main Lean check targets are:

```
lake env lean Axioms.lean      lake env lean Gates.lean
lake env lean TCB.lean         lake env lean Hardness.lean
lake env lean Proximity.lean
```

The full build target remains `lake build`; the smaller targets above are the theorem-surface checks most directly corresponding to this paper.

B Lean/Paper Correspondence

B.1 Definitions And Interfaces

HCP public observation and hidden witness.
Sections 2–3; `Hardness.PublicProjection`, `Hardness.HiddenOperatorFiber`, and `Hardness.HiddenOperatorClass`.

Kernel ambiguity and lossy public data.
Sections 2–3; `Hardness.KernelAmbiguous` and `Hardness.LossyPublicProjection`.

Explicit-lattice CVP comparison.
Section 3; `CVP.CVP_Instance`, `CVP.CVPSolution`, and `CVP.CVPOracleCorrect`.

Hodge projection model.
Sections 2–3; `Laplacian.harmonicProjection`, `Laplacian.residual`, and `Laplacian.IsHarmonic`.

Semantic/public equality surface.
Section 4; `Axioms.HelferFOL`, `Axioms.HomotopyClass`, and `Axioms.Fibration`.

Heyting/oracle public judgment.
Section 4; `Axioms.Heyting3`, `Axioms.OracleQuery`, `Axioms.OracleAnswer`, and `Axioms.OracleEta2`.

Cochain commitment payload.
Section 5; `Axioms.CochainCommitmentData`, `CochainCommitmentData.WellFormed`, and `CochainCommitmentData.AddCompatible`.

Commitment security interface.
Section 5; `Axioms.CochainCommitmentAxioms`.

Proof-carrying public artifact.
Section 5; `TCB.HashedCommittedCochain` and `TCB.ProofCarryingCommittedCochain`.

Projection chain.
Section 6; `ProjectionChain.Chain` and `ProjectionChain.Chain.toEvaluationProjection`.

Proximity-stage certificates.
Section 6; `Proximity.Gates.SourceMetricGate`, `Proximity.Gates.SyndromeAdequacyGate`, and `Proximity.Gates.HardGateCertificate`.

Canonical RS/MCA endpoint.
Section 6; `CanonicalRS.BridgeData`, `CanonicalRS.MCA.Line`, and `CanonicalRS.MCA.badChallengeFraction`.

B.2 Theorems And Status

Kernel ambiguity implies lossy public projection.
`Hardness.kernelAmbiguous_implies_lossy`; Lean-proved.

Public-compatible Hodge ambiguity gives kernel ambiguity.
`Hardness.kernelAmbiguous_of_publicCompatibleHodgeAmbiguity`; Lean-proved.

Hodge solver exposes a known-lattice CVP surface.
`Hardness.Hodge_Hardness_Reduction` and `Hardness.Hodge_Hardness_Reduction_Proof`; Lean-proved surface.

Known-lattice CVP does not select the hidden operator.
`Hardness.CVP_Oracle_Insufficient_For_Public_Hodge_Solving`; Lean-proved surface.

Projection-model Hodge recovery is CVP-shaped.
`Hardness.Hodge_Projection_Is_CVP` and `Hardness.harmonicProjection_Is_ProjectionHodg`
Lean-proved.

Exact sheaf gates supply $d^2 = 0$ and Laplacian-from-coboundary.
`Gates.structural_preconditions_of_sheaf_exact_gates`; Lean-proved alias/interface.

Well-formed payloads have nonempty edge data and supported modulus.
`CochainCommitmentData.edge_values_nonempty_of_wellFormed` and `CochainCommitmentData.modulus_gt_one_of_wellFormed`; Lean-proved.

Proof-carrying artifacts expose the committed payload.
`TCB.proofCarrying_cochain_wellFormed`; TCB boundary theorem.

Projection chain composes to one evaluation projection.
`ProjectionChain.Chain.toEvaluationProjection`; Lean-proved plumbing.

Syndrome adequacy turns zero local syndrome into codeword/global candidate.
`Proximity.Gates.SyndromeAdequacyGate.codeword_of_wellFormed_zeroSyndrome`; stage-local interface.

Final proximity evidence yields calibrated end-to-end readiness.
`Proximity.Gates.CalibratedFinalCertificate.toCalibratedEndToEndReady`; Lean-proved plumbing.

Canonical RS list/MCA bounds enter at a cited boundary.
`CanonicalRS.CitedBounds.bounds`; cited external theorem package.

Canonical MCA bad-challenge fraction is bounded after projection.
`CanonicalRS.MCA.projectionBadChallengeFraction_le`; Lean-proved from cited bounds.

References

[Hel19] Adam Helfer. First-order homotopical logic. *arXiv preprint*, 2019.